

# 1.2 主机内部硬件及工作原理

## 一、主存储器的基本组成与工作原理

主存储器用于存放数据和程序指令，其核心由**存储体**和两个关键的寄存器（**MAR**、**MDR**）组成。

### 1. 核心部件说明

| 部件简称 | 全称      | 功能描述                                      |
|------|---------|-------------------------------------------|
| 存储体  | 存储体     | 由一系列存储元件构成，用于实际存放二进制代码（0或1）。              |
| MAR  | 存储地址寄存器 | 存放与 <b>地址</b> 相关的二进制数据。指明当前要访问的存储单元地址。    |
| MDR  | 存储数据寄存器 | 存放 <b>实际的数据</b> 。无论是读出的数据还是准备写入的数据，都暂存于此。 |

### 2. 读写操作原理（以CPU访存为例）

- 读操作流程：
  - CPU 将想要读取的**数据地址**写入 **MAR**。
  - 主存储器的控制逻辑根据 MAR 接收到的地址，去存储体中找到对应的数据。
  - 将找到的数据先写入 **MDR**。
  - CPU 通过数据总线从 MDR 中取走它想要的数据。
- 写操作流程：
  - CPU 将想要写入的**具体位置（地址）**写入 **MAR**。
  - CPU 将想要写入的**具体数据**放入 **MDR**。
  - CPU 通过控制总线告诉主存储器此次执行的是**写操作**。
  - 主存储器根据这三个信息，将 MDR 中的数据写入到 MAR 指定的存储单元中。

### 3. 存储体的内部结构与相关概念

- 存储元件**：用于存储二进制数据的电子元件（利用电容原理，一个电容存放一个二进制比特位）。
- 存储单元**：由多个存储元件及相应线路构成，每个存储单元对应一个地址（如：地址 0、地址 1）。
- 存储字**：每一个存储单元存放的一串二进制代码。
- 存储字长**：每个存储单元可以存放的二进制位（比特）的数量，通常是 8 的整数倍（如 8、16、24）。

#### 💡 核心考点：MAR与MDR的位数含义

- MAR 的位数**：决定了存储体内有多少个存储单元。
  - 例如：若 MAR 有 4 个比特位，则存储体内最多有  $2^4 = 16$  个存储单元。
- MDR 的位数**：必须与存储单元保持一致，即**MDR 位数 = 存储字长**。
  - 例如：若 MDR=16 位，说明一个存储字的大小是 16 个比特。

#### ⚠️ 单位区分：

- 字节 (Byte)**：通常用大写 **B** 表示，1 字节 = 8 比特 (bit)。
- 比特 (bit)**：通常用小写 **b** 表示，表示 1 个二进制位。
- 字 (Word)**：一个字包含多少比特取决于计算机的硬件结构（可能是 8, 16, 32, 64 等）。

## 二、运算器的基本组成

运算器主要用来实现算术运算（加减乘除）和逻辑运算，包含以下核心部件：

| 部件名称   | 简称  | 功能描述                                        |
|--------|-----|---------------------------------------------|
| 算术逻辑单元 | ALU | <b>运算器的核心部件</b> ，集成复杂电路，实现各种算术和逻辑运算，制造成本最高。 |

| 部件名称  | 简称  | 功能描述                                |
|-------|-----|-------------------------------------|
| 累加器   | ACC | 用于存放操作数或运算结果。                       |
| 乘商寄存器 | MQ  | 只在乘法和除法运算时使用，存放乘除法的操作数或运算结果的高/低位。   |
| 通用寄存器 | X   | 用于存放通用操作数。（一般有多个，但理论上只需1个即可实现运算功能）。 |

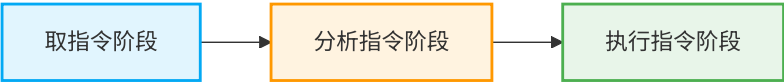
### 三、控制器的基本组成

控制器负责解析指令，并指挥其他部件协调工作：

| 部件名称  | 简称 | 功能描述                                        |
|-------|----|---------------------------------------------|
| 控制单元  | CU | 控制器的核心部件，集成复杂电路。完成分析指令，并给其他部件发出控制信号。制造成本极高。 |
| 指令寄存器 | IR | 用来存放当前正在执行的那条指令。                            |
| 程序计数器 | PC | 用来存放下一条指令的地址，并且具备自动加一的功能。                   |

### 四、计算机执行指令的三个阶段（取、析、行）

计算机的核心功能是执行代码（即一系列指令）。每完成一条指令，通常分为三个阶段：



- 1. 取指令：根据 PC 记录的地址，从主存中取出指令，放入 IR。随后 PC 自动加 1。
- 2. 分析指令：将 IR 中的操作码部分送入 CU，CU 分析这条指令的具体功能（加减乘除还是取数存数等）。
- 3. 执行指令：CU 控制其他部件（MAR、MDR、ALU、各种寄存器）配合完成指令的具体操作。

★ 注意：

对于任何指令，前两个阶段（取指令和分析指令）的执行步骤是一模一样的。只有分析完之后，CU 才知道要做什么，接下来的执行阶段才会因指令不同而有所区别。

### 五、指令执行完整过程剖析（以 $y = a * b + c$ 为例）

假设程序及变量已存入主存（字长16位，指令=操作码+地址码）。变量地址：a(5), b(6), c(7), y(8)。  
初始状态：PC = 0，准备读取第0条指令。

(说明：在寄存器外加括号如 (PC) 表示该寄存器内的值/内容)

#### 1. 第一条指令：取数指令（将变量 a 读入 ACC）

1. 取指阶段：

- (PC) -> MAR：PC的值(0)送入MAR，指明要访问0号单元。主存发起读操作。
- 主存将0号单元的数据放入 MDR。
- MDR -> IR：将 MDR 中的指令放入 IR。此时指令已取出。

2. 分析阶段：

- IR(操作码) -> CU：将指令前6位送到 CU 分析，得知这是一条“取数指令”。
- 此时 PC 自动加1：(PC) = 1。

3. 执行阶段：

- IR(地址码) -> MAR：将地址码(5)送入 MAR，指明要读取5号单元（变量a）。
- 主存将5号单元的数据读入 MDR。
- MDR -> ACC：将 MDR 中的数据(a的值，即2)送入累加器 ACC。

- 结果：(ACC) =  $a = 2$ 。

## 2. 第二条指令：乘法指令（计算 $a * b$ ）

1. 取指阶段：(PC) -> MAR，读取1号单元数据至 MDR，再送入 IR。
2. 分析阶段：送 CU 分析得知是“乘法指令”。PC 自动加1：(PC) = 2。
3. 执行阶段：

- IR(地址码) -> MAR：地址码(6)送入 MAR，指明读取变量 b。
- 主存将6号单元数据读入 MDR，随后 MDR -> MQ（将 b 放入乘商寄存器）。
- 将 ACC 中的 a 移入通用寄存器 X，即 (ACC) -> X。

底层做乘法时，ACC 必须先被清零，用来在每一轮计算中累加中间结果（部分积）

*X* 寄存器：固定存放被乘数  $a$ （整个计算过程中保持不变，每次累加都读取它）

*MQ* 寄存器：初始存放乘数  $b$ 。随着计算进行，低位一位位移出参与判断，移出的空位用来接收乘积的低位。

*ACC* 寄存器：初始置 0。每一轮将 *X* 加到 ACC 中（或加 0），然后连同 MQ 一起右移，最终存放乘积的高位。

把  $a$  移入 X，本质上是“给乘法运算腾出草稿纸”——让 *X* 专门锁定被乘数，好让 ACC 清空出来充当累加器。

- CU 指挥 ALU 将 (X) 与 (MQ) 相乘。
- 结果存回 ACC。如果结果过大，低位会存放在 MQ 中。

在十进制中，两个 2 位数相乘，结果最多是 4 位数；在二进制中也是同理：两个  $n$  位 (bit) 的数相乘，其乘积最长可能达到  $2n$  位。例如：两个 16 位的整数相乘，结果最多需要 32 位来存放。

CPU 内的每个通用寄存器（如 ACC、MQ）都有固定长度（例如都是 16 位或 32 位）。为了不丢失精度，硬件设计让 ACC 与 MQ 拼在一起组成一个双倍长度的寄存器池：

- 结果：(ACC) =  $a * b = 6$ 。

## 3. 第三条指令：加法指令（计算 $a * b + c$ ）

1. 取指阶段：(PC) -> MAR，读取2号单元数据至 MDR，再送入 IR。
2. 分析阶段：送 CU 分析得知是“加法指令”。PC 自动加1：(PC) = 3。
3. 执行阶段：

- IR(地址码) -> MAR：地址码(7)送入 MAR，读取变量 c 至 MDR。
- MDR -> X：将加数 c 放入通用寄存器 X。被加数( $a*b$ )已在 ACC 中。
- CU 指挥 ALU 将 (ACC) 与 (X) 相加。
- 结果再次存回 ACC。
- 结果：(ACC) =  $a * b + c = 7$ 。

## 4. 第四条指令：存数指令（将结果赋给 y）

1. 取指阶段：(PC) -> MAR，读取3号单元数据送入 IR。
  2. 分析阶段：送 CU 分析得知是“存数指令”。PC 自动加1：(PC) = 4。
  3. 执行阶段：
- IR(地址码) -> MAR：将地址码(8)送入 MAR，指明写入位置（变量y）。
  - (ACC) -> MDR：将运算结果(7)送入 MDR，准备写入。
  - CU 通过控制线通知主存执行写操作。主存将 MDR 数据写入8号单元。
  - 结果：变量  $y$  的存储位置被写入了最终结果 7。

## 5. 第五条指令：停机指令

1. 取指、分析后，CU 发现是“停机指令”。
2. 此时硬件执行的流程结束，转由操作系统通过系统调用或中断机制来终止该进程。

# 六、核心总结补充

### 1. CPU 如何区分指令和数据？

- 在取指令阶段：主存取出的内容会被送到 IR (指令寄存器)，此时认定为指令。
- 在执行指令阶段：根据 CU 的控制，主存取出的内容会被送到 ACC / MQ / X 等寄存器，此时认定为数据。
- 即：根据指令执行周期所处的不同阶段来区分。

## 2. 主机概念补充：

- 主存、运算器、控制器共同构成了**主机**。
- 逻辑上，MAR 和 MDR 属于主存；但在**现代计算机中，这两个寄存器通常被集成在 CPU 内部**。

## 3. 同等地位与地址寻访：

- 指令和数据无差别地以同等地位放到存储器中，只是存放位置不同。
- 无论是读指令还是读数据，都必须给出其在内存中的地址，这叫做**按地址寻访**。

## 4. 多地址指令：

- 有的计算机支持一条指令包含多个地址码。例如包含两个地址码的指令被称为“二地址指令”。

## 5. 存储程序概念：

- 再次强调：在程序运行之前，指令和数据都会被提前存到主存里面。